

| THE VAULT • EDITION 01

The Prompt Vault.

28 production prompts from the teams shipping AI at scale. **The text. The system. What breaks.**

28

What's in here, what's **not**.

Twenty-eight prompts. Five sections. Five short stories between them about the people who run these prompts and what happens when the prompts go wrong.

Every prompt is sourced from a real, public, citable place: Anthropic's official documentation, the publicly-mirrored system prompts of shipped products like Cursor and Claude Code, the published work of named engineers who put their prompts on the internet.

Nothing in here is invented. The sources are listed on the next page.

The argument: the prompt is the smallest part of why production AI systems work. The other **80%** — the structured output, the negative instructions, the failure modes, the observability — is what nobody is writing about. This book takes 28 prompts apart so you can see all four layers.

Read in order. The pages are short. The stories matter as much as the prompts.

— Mason

Every prompt cited.

ANTHROPIC · OFFICIAL DOCUMENTATION

Prompting Best Practices Guide

– platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Customer Case Studies — StubHub, Decagon, and others

– claude.com/customers

Use Case Guides — Ticket routing, legal summarization, customer support

– platform.claude.com/docs/en/about-claude/use-case-guides

ANTHROPIC · ENGINEERING

Building Effective Agents

– anthropic.com/engineering/building-effective-agents

LEAKED & MIRRORED PRODUCTION SYSTEM PROMPTS

Claude Code v2.1.146 system prompts

– github.com/Piebald-AI/claude-code-system-prompts

Cursor, v0, Devin, Windsurf, Lovable, Replit, Bolt, Manus, Perplexity

– github.com/elder-plinius/CL4R1T4S

Additional v0 / Cursor / Devin / Replit Agent

– github.com/lucasmrtd/leaked-system-prompts

NAMED ENGINEERS · PUBLISHED WORK

Eugene Yan — LLM patterns, evals, applied ML – eugeneyan.com

Simon Willison — Working notes on LLM tooling – simonwillison.net

Hamel Husain — LLM eval methodology – hamel.dev

DOCUMENTED PRODUCTION PATTERNS

Customer support automation — eesel AI, Swrly, Bland AI tutorials

Where a prompt is a constructed pattern built from these sources rather than a verbatim lift, it's marked **Pattern · constructed**. The architecture is real; the exact text is our composition.

Everyone is teaching you the wrong part.

It's 11pm. You're scrolling.

Someone posted a prompt that promises to 10x your productivity. You save it. You'll never use it.

Below it: a thread on the "five prompt frameworks every founder should know." You save the thread. You won't read it again.

Below that: an ad for a course about how to write prompts that "actually work."

You close the app feeling vaguely behind. **You're not.** You're being sold to by an industry that needs you to feel that way.

Meanwhile, in a customer service queue at StubHub, the world's largest live event ticketing marketplace, a prompt is quietly running. It doesn't have a clever framework. It's not a one-liner. It's a system prompt connected to a knowledge base, customer details, and order context, then it resolves tickets in seconds. The same system that used to take customers 20 minutes of wait time and multiple phone calls.

The result, per Anthropic's published case study: **wait times collapsed from over 20 minutes to near-instant. Support costs dropped by more than 30%.**

The prompt is not the magic. The prompt is the smallest part.

The magic — if you want to call it that — is in four things you do not see when someone screenshots a prompt and posts it.

- 01 **The output is a structured object.** Not a paragraph. A predictable shape with named fields. The next system in line can read it without parsing English.
- 02 **Most of the lines are negative instructions. Do not invent. Do not extrapolate. Do not synthesize.** The positive instructions describe the dream. The negatives are the actual work.
- 03 **The prompt knows how it fails.** The engineers who built it have already met its worst inputs. There is documentation, often in the prompt itself, of what breaks and what the system does about it.
- 04 **The output shows its work.** Confidence scores. Reasoning fields. A list of signals used. When the model is wrong, the team can read why.

Four pieces. **Structure. Negatives. Failure modes. Observability.**

That is the 80% nobody is writing about.

This book is twenty-eight prompts torn apart so you can see all four pieces on every page.

But before each section, there is a story. Five of them. Each one is the story of a real-feeling moment: the founder who shipped the wrong auto-reply at 11pm. The CMO whose engagement numbers were dropping. The buyer who trusted a comparison matrix and regretted it. Each story is short — two pages. Each one names what the next six prompts are really for.

The prompts are real. Three are verbatim lifts from Anthropic — two from the public mirror of Claude Code's system prompts (Piebald-AI/claude-code-system-prompts), one from Anthropic's published prompting best practices guide. Two are adapted from Anthropic's prompting guide with editorial additions named explicitly on the page. The rest are patterns — constructed from techniques documented by named engineers and from publicly-mirrored system prompts in the CL4R1T4S repository (Cursor, v0, Devin, Lovable, and others).

Read in order. The stories are not optional. They are the part that tells you why the prompts matter. Halfway through the book there is a chapter called **Why the prompt is the smallest part**. Read it slowly. The prompts look different on the other side of it.

If you take one thing from this book, take this: **the prompts that work in production are prompts that have been broken in production first.**

— Mason

| SCENE ONE / BEFORE SECTION I

Lena's 2:14am ticket.

TUESDAY · 7:47 AM

It's 7:47am on a Tuesday in October, and Lena is at her desk before her team.

The Slack channel is already on fire. Six tickets in the support queue since midnight. One of them — third from the top, posted at 2:14am, no priority tag — is from the largest customer her company has. **She doesn't know that yet.** She's still drinking the coffee.

The previous version of Lena would have spent the next 40 minutes triaging by hand. Reading subject lines. Scanning bodies. Tagging in Slack. **The previous version of Lena is the one she's trying to retire.** The version of Lena at this desk has, sitting in a file on her laptop, twenty lines of text that do the triage for her.

She pastes the 2:14am ticket into the chat window. She hits enter. Three seconds pass.

RESPONSE

```
{ "priority": "P1", "channel": "#support-p1", "confidence":  
0.94, "reasoning": "customer reports billing block on  
enterprise plan; explicit mention of revenue impact;  
escalation language" }
```

Lena posts it to the channel.

| SCENE ONE / CONTINUED

This is the part of the story they screenshot. The twenty-line prompt. The fast result. The early morning win.

This is not the story.

The story is the six hours Lena spent last week building the prompt. The 50 historical tickets she ran it against. The three categories of failure she found — Enterprise customers with personal Gmail addresses getting downgraded, long ticket bodies anchoring on the last paragraph, frustrated tone leaking into priority decisions. The four lines she added to the prompt to fix each one. The shadow-mode week where the model's classifications ran silently alongside the human triage, and she compared them.

The prompt that runs in three seconds is the artifact. **The system that lets Lena trust it is the work.**

That distinction — the prompt as artifact, the work as the system around it — is the whole point of this book.

The next six prompts are real. They will work on the easy cases. They will fail in the interesting ways the **What Breaks** section describes. **Read those sections first.**

The artifact is short. The work is long. That is the point.

The obvious part.

Support triage. Repository docs. Meeting extraction. Lead routing. Standup summaries. Email triage.

The bread and butter. Copy them. Run them. They will work on the easy cases and fail in interesting ways on the hard ones.

That is the point. Read the What Breaks section on each page first.

01

SECTION 01 · OPERATIONS

PROMPT 01 OF 28

LENA'S PROMPT.

| USE CASE

Classifies incoming support tickets by priority (P1–P4) and routes them to the right Slack channel without a human in the triage loop. This is the prompt from the scene before this section.

| THE PROMPT

PROMPT

You are a customer support triage specialist. You receive incoming support tickets and classify them by priority.

Read the ticket subject, body, customer plan, and any context provided. Assign exactly one priority level:

- P1 (Critical): Service is down, data loss, security incident, or billing blocked for Enterprise customers. Immediate action required. Route to #support-p1.
- P2 (High): Significant functionality broken for paid customers, or workaround exists but is painful. Route to #support-p2. Target response: 4 hours.
- P3 (Medium): Bug or friction not blocking core workflow. Route to #support-p3. Target response: 24 hours.
- P4 (Low): Feature request, question, or cosmetic issue. Route to #support-backlog. Target response: 3 days.

Return JSON:

```
{
  "priority": "P1" | "P2" | "P3" | "P4",
  "channel": "#support-p1" | "#support-p2" | "#support-p3" | "#support-backlog",
  "reasoning": "one sentence explaining your decision",
  "confidence": 0.0-1.0
}
```

If the ticket is ambiguous, default to one level higher than your best guess. Over-classifying a P3 as a P2 wastes a human glance. Under-classifying a P1 as a P2 burns trust.

| WHAT IT POWERS

A drop-in classification step at the front of a support workflow. JSON output feeds a router that posts to Slack and creates the ticket in the helpdesk with priority pre-tagged.

| WHAT BREAKS

- We learned this on day three: free-tier users reporting genuine outages get under-classified as P3 because the prompt over-weights "Enterprise" as a P1 signal. Add: **a real outage is P1 regardless of plan.**
- Confidence below 0.7 should route to a human queue, not the predicted channel. The model will return 0.4 confidence and a "P2" decision in the same response. The router will happily fire it.
- Long ticket bodies cause the model to anchor on the last paragraph and miss the actual incident higher up.
- Frustrated tone leaks into priority. Tone is not severity.

| SOURCE

Pattern · constructed from production support automation tutorials (eesel AI, Swrly, Bland AI). The "default to one level higher" mechanism is the load-bearing instruction.

Without the **"default to one level higher"** line, models bias toward the safe middle and your P1 queue is empty until something is on fire. Test against your last 50 tickets before shipping anything. Lena did. **You should too.**

DEMONSTRATES **structured output** • **failure modes**

02

FEATURED

The **repository bootstrap** — Claude Code's own prompt for documenting a new codebase.

SOURCE: PIEBALD-AI/CLAUDE-CODE-SYSTEM-PROMPTS · AGENT-PROMPT-CLAUDEMD-CREATION.MD

| THE PROMPT

PROMPT

Please analyze this codebase and create a CLAUDE.md file, which will be given to future instances of Claude Code to operate in this repository.

What to add:

1. Commands that will be commonly used, such as how to build, lint, and run tests. Include the necessary commands to develop in this codebase, such as how to run a single test.
2. High-level code architecture and structure so that future instances can be productive more quickly. Focus on the "big picture" architecture that requires reading multiple files to understand.

Usage notes:

- If there's already a CLAUDE.md, suggest improvements to it.
- When you make the initial CLAUDE.md, do not repeat yourself and do not include obvious instructions like "Provide helpful error messages to users", "Write unit tests for all new utilities", "Never include sensitive information (API keys, tokens) in code or commits".
- Avoid listing every component or file structure that can be easily discovered.
- Don't include generic development practices.
- If there are Cursor rules (in .cursor/rules/ or .cursorrules) or Copilot rules (in .github/copilot-instructions.md), make sure to include the important parts.
- If there is a README.md, make sure to include the important parts.
- Do not make up information such as "Common Development Tasks", "Tips for Development", "Support and Documentation" unless this is expressly included in other files that you read.

| WHAT IT POWERS

The first session on any new codebase inside Claude Code. The resulting CLAUDE.md is loaded at the start of every future session, replacing 5-10 minutes of rediscovery work with a small reference file.

| WHAT BREAKS

- On large monorepos the model attempts to summarize every package and produces a 4,000-word document that no future session will actually load.
- Obvious instructions sneak back in despite the explicit ban. Reject the first draft if it includes anything an experienced engineer would already know.
- Stale almost immediately. The CLAUDE.md from January describes a codebase that no longer exists by March.

Count the **Usage notes** bullets. Five of seven are negative instructions — "do not repeat," "avoid listing," "do not include," "do not make up." Two are positive ("if there's already...", "if there are Cursor rules..."). **The negative-to-positive ratio is the tell.** This is the pattern under almost every prompt in this book, and we will name it explicitly two sections from now.

DEMONSTRATES **negative instructions**

03

| USE CASE

Reads a long meeting transcript and extracts only the action items, with assignee and deadline, ignoring everything else.

| THE PROMPT

PROMPT

You are extracting action items from a meeting transcript.

An action item is a specific task that:

1. A named person committed to doing
2. Has a clear deliverable (something exists or changes after it is done)
3. Has either an explicit deadline or a reasonable default of "by next meeting"

Things that are NOT action items:

- General discussion or opinions
- Decisions made (these go in a different document)
- Suggestions no one took ownership of
- Anything prefaced with "we should probably..." unless someone then said "I'll do that"

For each action item, return:

- assignee: the person's name (exact form from transcript)
- task: one sentence, starts with a verb
- deadline: explicit date if given, otherwise "by next meeting"
- source: the verbatim quote where the commitment was made

Return as a JSON array. If there are no action items, return an empty array []. Do not invent action items to fill the array.

| WHAT IT POWERS

The post-meeting automation that turns a 45-minute call transcript into a list. Sits downstream of any transcription service (Otter, Fireflies, Granola, Read.ai) and upstream of a project tool API.

| WHAT BREAKS

- "We should probably look into that" gets converted to "Look into that — assignee: unknown" despite the instruction. Models hate empty arrays.
- Multiple speakers using "I'll handle it" without a clear referent — the model assigns to whoever spoke last.
- Soft commitments ("I can try to") become hard action items. The source-quote field is your audit trail to catch this.
- Non-English names get mangled.

| SOURCE

Pattern · constructed from Anthropic's structured extraction guidance. The "source quote" requirement turns this from a list into an artifact you can audit.

The "**do not invent action items to fill the array**" line is the most important sentence. Models will pad. The verbatim source quote is your only defense. Make the reviewer check it.

DEMONSTRATES **structured output** · **observability**

04

| USE CASE

Parses an incoming sales lead form submission and routes it to one of five queues: self-serve, SMB, mid-market, enterprise, or disqualified.

| THE PROMPT

PROMPT

You are a lead routing specialist. Read the form submission and decide which sales queue it belongs in.

Tiers:

- self_serve: Sole proprietors, individuals, or companies under 10 employees.
- smb: 10-100 employees.
- mid_market: 100-1000 employees.
- enterprise: 1000+ employees, or any company in the named accounts list.
- disqualified: Competitor, student/academic with no budget signal, or obvious spam.

Signals in order of weight:

1. Explicit headcount field if provided
2. Email domain (gmail/yahoo = likely individual; custom domain = company)
3. Company name lookup against named accounts list
4. Job title
5. Use case described in free text

Return JSON:

```
{
  "tier": "self_serve" | "smb" | "mid_market" | "enterprise" | "disqualified",
  "confidence": 0.0-1.0,
  "signals_used": ["list", "of", "signals"],
  "disqualification_reason": "string or null"
}
```

If confidence < 0.6, set tier to "smb" as a safe default and flag for human review.

| WHAT IT POWERS

Drop-in step between Typeform/HubSpot intake and CRM assignment rules. Replaces the SDR who used to spend 20 minutes a day eyeballing the queue.

| WHAT BREAKS

- Personal Gmail addresses from real enterprise buyers (CTO using gmail) get downgraded to self_serve.
- "We're a small team but enterprise-grade" copywriting confuses the model.
- Companies with generic names ("Apex," "Summit," "Vertex") trip false positives against the named accounts list.
- First week in shadow mode, the model disqualified a \$40K agency partner as a "competitor." The disqualification tier is almost always wrong: it flags partners, agencies, and consultants who are actually qualified buyers.

| SOURCE

Pattern · constructed from Anthropic's ticket routing use case guide, adapted from support to sales.

The **safest fix is to remove the disqualification tier entirely**. Let humans disqualify. The cost of a missed lead is higher than an SDR glance.

DEMONSTRATES **structured output** · **observability** · **failure modes**

05

| USE CASE

A daily standup summarizer: ingests the morning's Slack channel posts from a team and produces a single summary message — what each person is working on, blocked on, and shipped yesterday.

| THE PROMPT

PROMPT

You are summarizing a team's morning standup posts from Slack. The posts are below in chronological order, each labeled with the author's name.

Produce a single summary message in this exact format:

 Standup — [today's date]

Shipped yesterday

- [name]: [one-line summary]

Working on today

- [name]: [one-line summary]

Blocked / needs help

- [name]: [blocker description] — [who can unblock if mentioned]

Mentions / cross-team

- [thing that needs someone outside this team's attention]

Rules:

- One line per person per section. If they had three things, pick the most important.
- Use exact names as they appear in the posts.
- If someone didn't post, do not include them and do not flag the absence.
- If no one is blocked, omit the "Blocked" section entirely.
- Do not editorialize or add encouragement. Just the facts.

| WHAT IT POWERS

A scheduled job that runs at 10:30am, reads the #standup channel from 8–10am, and posts the summary to a #leads-only channel.

| WHAT BREAKS

- People who post async outside the time window get dropped silently.
- Emoji-heavy posts ("🚀🚀🚀 SHIPPED!!!") get parsed as low-information.
- Blockers phrased as questions don't get flagged as blockers.
- Thread replies aren't included — if blocker resolution happened in a thread, the summary still says blocked.

| SOURCE

Pattern · constructed from documented summarization patterns in Anthropic's prompt library and the structured-output enforcement in the leaked Cursor system prompt (CL4R1T4S).

Resist the urge to add sentiment analysis. The leads don't need **"team morale: 7/10."** They need to know who's stuck.

DEMONSTRATES **negative instructions**

06

SECTION 01 · OPERATIONS

PROMPT 06 OF 28

— THE DANGEROUS ONE.

Reads a long inbound email and decides whether it requires a human reply, can be auto-replied, or should be **archived.**

| THE PROMPT

PROMPT

You are an email triage assistant. For each incoming email, decide one of three actions:

1. HUMAN_REPLY: The sender expects a personal response.
2. AUTO_REPLY: A standard template will address this completely. Choose from "received_application", "out_of_scope", or "scheduling".
3. ARCHIVE: No response needed. Newsletters, notifications, FYI emails, automated reports.

Read the full email body. Do not decide based on the subject line alone.

Important signals:

- A question mark in the body usually means HUMAN_REPLY.
- "Quick favor" / "got a minute" usually means HUMAN_REPLY.
- "Unsubscribe" link in the footer with no personal greeting usually means ARCHIVE.
- "Looking forward to your response" means HUMAN_REPLY regardless of the rest of the content.

Return JSON with action, template_key, confidence, reasoning.

When uncertain, choose HUMAN_REPLY. A missed reply is worse than an unnecessary human glance.

| WHAT IT POWERS

Sits in front of an executive's inbox via the Gmail API. Routes ARCHIVE items out of sight, AUTO_REPLY items to a queue with a 4-hour delay, HUMAN_REPLY items to the top.

| WHAT BREAKS

- Calendar invites get archived as "automated" when they need a yes/no.
- Personal emails written in a transactional tone get archived.
- Late-night auto-replies ship before the human wakes up to override.

This is the **highest-stakes prompt in this section**. An auto-reply to your largest customer addressed as "out_of_scope" will end a contract. **Shadow mode for at least a week before letting it send anything**. The story before Section IV is about a founder who didn't.

| SOURCE

Pattern · constructed from Anthropic's customer support use case guide and the documented triage pattern in eesel AI's production support tutorials.

DEMONSTRATES

structured
output · failure
modes · observability

Structured output is the contract.

Every prompt in the section you just read returned **a shape, not a paragraph**. A JSON object with named fields. An action item array. A summary in a fixed template.

That is not a stylistic preference. That is the contract between the model and the system that has to do something with the model's output. A paragraph is for a human reader. A shape is for a machine that has to route the ticket, populate the project tool, post to the right channel.

When you write a prompt that returns prose, you are saying: **a human will read this and decide what to do**. When you write a prompt that returns structure, you are saying: **this output enters a pipeline**. The two are different products. Most "prompt advice" you've read on the internet is for the first one. **Production is the second one.**

Watch for the **confidence** field on the next twelve prompts. It is the model's offer to tell you when it should not be trusted. Most pipelines throw it away. **Don't.**

— Mason

| SCENE TWO / BEFORE SECTION II

The **argument** the CMO didn't realize he was having.

Three thousand miles away from Lena's office, a content team is having an argument they don't realize is the same argument.

The CMO wants more output. The writers want more time. The newest hire — a 24-year-old with a Substack and opinions — has solved the disagreement by quietly running every draft through ChatGPT for an extra polish before sending it up.

The polish is detectable. The reader can't name it, but they can feel it. The "**in today's fast-paced world**" creeping into the opening lines. The em-dashes carrying parenthetical asides they didn't need. The lists of three where the third item is "and scalability." The conclusions that begin with "**Ultimately.**"

It's not that the model is bad at writing. **The model is too good at average writing.** That's the problem.

The CMO doesn't know any of this. **The CMO is asking why the engagement numbers are dropping.**

| SCENE TWO / CONTINUED

The next six prompts are for content work, and they fight the model **differently than the operations prompts in Section I did.**

The operations prompts had a clear answer the model could be right or wrong about. Triage the ticket. Route the lead. Summarize the standup. **Right answers exist.**

Content has no right answer. Content has **defaults the model reaches for when nothing is stopping it**. The model has been trained on millions of examples of mediocre business writing. When you ask it to write, it writes toward the gravitational center of that training. The corporate-warm preamble. The hedge. The third item in the list.

The prompts that work for content are not the ones that describe the dream. The prompts that work are the ones that **tell the model which gravity wells to avoid**.

Watch the "do not" lists in this section. They are longer than the "do" lists. That is not a stylistic choice. That is the prompt.

When the words get **slippery**.

Editing. Voice. Slop detection. Long-form drafting. The orchestrator. The translator.

These prompts do not have right answers. They have **better and worse defaults**. The work is telling the model which defaults to abandon.

07

| USE CASE

Builds an outline for a long-form blog post from a one-line topic plus three constraints: audience, key point, banned angle. Produces a structure tight enough that the writing step is just expansion.

| THE PROMPT

PROMPT

You are a blog post architect. Given a topic and three constraints, produce a tight outline.

Topic: {TOPIC}

Audience: {AUDIENCE – be specific: "Series A startup CTOs," not "tech leaders"}

Key point: {ONE SENTENCE THE READER SHOULD AGREE WITH BY THE END}

Banned angle: {THE OBVIOUS TAKE WE'RE NOT TAKING}

Produce an outline with this structure:

1. Opening hook (2-3 sentences, no rhetorical questions, no "imagine if," no statistics from a study)
2. The setup (200 words: what's the situation that makes this worth reading?)
3. Three to five sections, each with:
 - Section heading (specific, not "Why X matters")
 - One sentence summary of the argument
 - Two to three concrete examples or evidence points
4. The turn (the moment in the piece where the obvious take is set aside and the actual argument begins)
5. The conclusion: state the key point in different words than the introduction
6. The action: one specific thing the reader should do

Rules:

- The outline should be specific enough that two different writers would produce structurally similar drafts from it.
- Do not pad with sections that say "background" or "context."
- Do not include a section titled "challenges" or "considerations."
- If you cannot honestly argue the key point without

the banned angle, say so explicitly. Do not smuggle the banned angle in.

| WHAT IT POWERS

First step in a content pipeline. The outline goes to a human writer or to a writer prompt for expansion. Used to enforce structural quality at the cheapest point in the process.

| WHAT BREAKS

- The "banned angle" rule is the hardest to enforce. Models will produce outlines that gesture at the banned angle in section 4 and then claim they avoided it.
- "Specific enough that two writers would produce structurally similar drafts" is a soft requirement that doesn't always land.
- Models pad section 2 every time. Cut it in half before sending to the writer.
- The "key point" sometimes gets diluted as the outline progresses.

| SOURCE

Pattern · constructed from documented content-architecture patterns in Anthropic's prompt library and adapted from outlining techniques published by named editors (Eugene Yan, eugeneyan.com).

The audience constraint is the lever that matters most. "Tech leaders" and "Series A startup CTOs" produce completely different outlines. **Be allergic to vagueness in that field.**

DEMONSTRATES **negative instructions**

08

| USE CASE

Critiques a draft against a specific brand voice guide, returning line-by-line edits with rationale. Used as a self-review step before a human reviewer.

| THE PROMPT

PROMPT

You are a writing editor. Below is a draft and a brand voice guide. Your job is to mark every line that violates the voice guide and propose a fix.

Voice guide:

```
<voice_guide>
{INSERT VOICE GUIDE – banned words, tonal anchors,
audience description, formatting rules}
</voice_guide>
```

Draft:

```
<draft>
{INSERT DRAFT}
</draft>
```

For each violation, return:

- line: the exact line as it appears
- violation: which rule was broken (use the exact rule language from the voice guide)
- proposed_fix: a rewrite of the line that fixes the issue without changing the meaning
- confidence: 0.0-1.0

Rules:

1. Do not propose stylistic preferences not in the voice guide. If you think a sentence is "too long" but length is not in the guide, leave it alone.
2. Do not flag the same violation twice if it appears multiple times in similar contexts. Mark the first instance and note "+ N more occurrences."
3. Do not propose fixes that change the factual content of the line.
4. If a line is correct, do not include it in the output.

Return as a JSON array of violations. Empty array if no violations found.

| WHAT IT POWERS

Pre-publication review step. Sits between the writer (human or AI) and the editor. Catches the obvious voice violations so the human editor can focus on the harder calls: argument, structure, accuracy.

| WHAT BREAKS

- Models drift toward proposing "more concise" fixes even when concision isn't in the voice guide. They were trained to value brevity.
- The "exact rule language" requirement frequently gets paraphrased.
- Sarcastic or ironic lines get flagged as tonal violations because the model reads them at face value.
- If the voice guide includes both "be warm" and "avoid hedging," the model produces contradictory edits on the same line.

| SOURCE

Pattern · constructed from Anthropic's prompt library editing examples and the XML-tagging conventions in the Anthropic prompting best practices guide.

The most useful output isn't the edits. It's the lines that get flagged repeatedly. Those tell you which voice rules are unclear in your guide.

DEMONSTRATES **negative instructions** · **observability**

09

| USE CASE

Generates 10 social-post variants from a single source article, each tuned for a different platform and a different angle.

| THE PROMPT

PROMPT

You are a social copywriter. Below is a source article.
Generate exactly 10 social posts:

- 3 for X (max 280 chars, no hashtags, no "thread 🧵" language)
- 4 for LinkedIn (max 1,200 chars, professional but not corporate, no "What I learned about..." openings, no "Here's the thing..." patterns)
- 3 for Threads (max 500 chars, conversational, more personal voice acceptable)

Across the 10 posts, use these angles (one per post):

1. Single insight (state the surprising finding)
2. Contrarian (challenge the obvious interpretation)
3. Quote pull (lift the strongest line verbatim)
4. Question (ask the reader something specific)
5. Numbered list (3-4 items max)
6. Personal stake (why this matters to one person)
7. Comparison (X is to Y as A is to B)
8. Behind-the-scenes (the messy reality)
9. Counter-example (what doesn't fit the rule)
10. Direct ask (tell the reader what to do)

Do not use:

- "Here's why this matters"
- "What most people miss"
- "Hot take:"
- "Unpopular opinion:"
- "Thread 🧵"
- Hashtags of any kind unless explicitly requested
- More than one emoji per post

Format your response as a JSON array with platform, angle, post, and char_count fields.

| WHAT IT POWERS

Common pattern in content teams: one long-form piece per week, 10 social posts distributed across the week. Runs in n8n or Zapier off a CMS trigger.

| WHAT BREAKS

- Character counts are inconsistent. Models miscount emoji and links. Always validate post-generation in code.
- The "contrarian" angle becomes "this is actually bad" on every topic.
- LinkedIn posts default to corporate-speak even with the explicit instruction.
- 10 posts is too many for most source articles. Most articles support 4–5 genuinely distinct angles. The rest will be filler.

| SOURCE

Pattern · constructed. The multi-variant generation structure is a common pattern in production content generation systems — v0, Cursor, and other shipped tools published in the CL4R1T4S repository all use variations of it.

Take the top 4 and throw the other 6 away. The model will give you all 10 happily; **that doesn't mean all 10 are usable.**

DEMONSTRATES **structured output** · **negative instructions**

10

SECTION 02 • CONTENT

PROMPT 10 OF 28

— THE SLOP DETECTOR.

Catches the tonal patterns that make AI writing recognizable as AI writing — the slop detector for prose.

| THE PROMPT

PROMPT

You are reviewing a piece of writing for "AI slop" – the tonal patterns that make AI-generated writing recognizable as such even when the facts are correct.

Flag any of the following patterns:

STRUCTURAL:

- Sentences that open with "In today's fast-paced world"
- Paragraphs that end with rhetorical questions the writer doesn't answer
- "Not just X, but Y" constructions used more than once
- Lists of three where the third item is generic ("speed, accuracy, and quality")
- "It's important to note that..." or "It's worth noting"
- Conclusions that begin with "In conclusion" or "Ultimately"

LEXICAL:

- "Delve into," "unpack," "explore," "navigate" (when used as throat-clearing verbs)
- "Robust," "comprehensive," "seamless," "cutting-edge"
- "Plays a crucial role" / "vital role" / "key role"
- "In the realm of"
- "Game-changer" / "game-changing"
- Em-dashes used to insert parenthetical asides more than twice per page

STRUCTURAL VOICE:

- Hedging language ("perhaps," "might," "could potentially") used in declarative contexts

- Paragraphs that summarize what was just said before moving on
- The phrase "this approach" used as the subject of a sentence
- Em-dashes acting as colons

For each instance found, return line, pattern, severity (high/medium/low), and suggested_rewrite.

Return as JSON array. Be merciless. False positives are cheaper than false negatives here.

| WHAT IT POWERS

Final-pass review on any AI-assisted draft before publication. Catches the patterns that read as AI to anyone who's been online for two years.

| WHAT BREAKS

- The model is generous about its own output. Won't flag "delve into" if that phrase is the most natural fit.
- Severity calibration drifts; everything starts to feel "medium."
- Em-dashes are over-flagged when the writer uses them correctly.

Run this on your own writing once.

The list of patterns it catches in you is informative. **We started this as an AI-detection tool and ended up using it as a personal style mirror.**

| SOURCE

Pattern · constructed from documented "AI tells" cataloged across multiple published sources (Simon Willison's working notes at simonwillison.net, Eugene Yan's writing on LLM output evaluation).

DEMONSTRATES **negative**
instructions · **observability**

11

SECTION 02 · CONTENT
PROMPT 11 OF 28
PATTERN FROM ANTHROPIC ENG.

| USE CASE

Generates a long-form draft from an outline using the orchestrator-workers pattern: a planning step produces the section list, then parallel writer steps draft each section.

| THE ARCHITECTURE

FLOW

- 1 **Outline input** — A single outline is handed to the orchestrator.
↓
- 2 **Orchestrator step** — Breaks the outline into independently-draftable sections. Writes a brief for each one: heading, argument, examples, transitions in/out.
↓
- 3 **Parallel writers** — N writer calls run simultaneously, one per section brief. Each writer drafts only its own section and is forbidden from summarizing the rest.
↓
- 4 **Stitch** — Sections concatenate in outline order. Transitions hold because the orchestrator wrote them, not the writers.

| THE PROMPT

PROMPT

[ORCHESTRATOR STEP]

You are the orchestrator for a long-form writing task.
Given the outline below, your job is to:

1. Break the writing into independently-draftable sections
2. For each section, write a writer brief that contains:
 - The section heading
 - The argument the section needs to make
 - 2-3 concrete examples the section must include
 - The transition INTO this section from the previous one
 - The transition OUT to the next section
3. Identify any sections that depend on other sections being drafted first (most don't)
4. Return a JSON object with the brief for each section

Outline:

```
<outline>{OUTLINE}</outline>
```

[WRITER STEP – runs in parallel for each section]

You are drafting one section of a long-form piece. Below is the brief for your section.

Brief:

```
<brief>{SECTION BRIEF}</brief>
```

Draft the section in {N} words. Use the examples in the brief. Match the transitions. Do not draft other sections. Do not summarize the rest of the piece. Do not add a section conclusion that previews the next section – the transition handles that.

| WHAT BREAKS

- Orchestrator gets the section breakdown slightly wrong and the writers produce sections that overlap or leave gaps.
- Transitions handed to writers are too vague ("connect from the previous section") and the writers each invent their own connection language.
- Writers default to summarizing the previous section in their opening line despite the instruction.
- Parallel cost is real. N writer calls per draft, where N is the number of sections.

| SOURCE

Pattern from Anthropic — "Building Effective Agents," anthropic.com/engineering/building-effective-agents. The orchestrator-workers pattern is theirs; the specific two-step prompt construction shown here is Mason's. **One of the most-cited engineering posts of 2024–2025.**

This is the most underrated pattern in the book. Most teams reach for a single long prompt when they should be reaching for **an orchestrator and three workers**.

DEMONSTRATES **structured output**

12

| USE CASE

Cuts a draft by a specified percentage without changing the argument. Used when a piece comes in long and the writer needs help finding what to remove.

| THE PROMPT

PROMPT

You are cutting a draft. The current draft is below.
Target length: {N} words (currently {M} words).

Your job is to remove material without:

- Changing the argument or conclusion
- Cutting any specific examples, quotes, or numbers
- Cutting any line the author flagged with [keep]
- Removing transitions to the point of incoherence

Things to cut first:

1. Sentences that summarize what was just said
2. "It's worth noting that" / "It's important to" / "Of course" / "Naturally" preambles
3. Adverbs that don't change meaning ("really," "actually," "very," "quite")
4. Hedges that aren't load-bearing ("might possibly," "could potentially," "in some cases")
5. Sentences that restate the section heading
6. The second example when two examples make the same point
7. The transition paragraph between sections (often the section heading does this work alone)

Things to cut last:

1. Direct quotes
2. Concrete numbers
3. Named examples
4. Counterarguments and concessions
5. The opening hook
6. The closing line

Return the cut draft and a list of what was removed, with text and reason for each removal. If you cannot hit the target length without violating the "do not cut" rules, return the closest you can get and flag it.

| WHAT IT POWERS

The "cut to length" step in any content workflow with hard length constraints: newsletters with character limits, executive summaries, social posts. Replaces the painful manual pass where the writer cuts their own favorite lines.

| WHAT BREAKS

- The model preserves "examples" so liberally that the cut never quite hits the target.
- Transition paragraphs get cut too aggressively; the resulting draft reads choppy.
- The "removed" log is useful for editor review but doubles the output token count.
- Quotes longer than 30 words sometimes get paraphrased "to save space" despite the explicit rule.

| SOURCE

Pattern · constructed. The "things to cut first / things to cut last" framing is adapted from editorial methodology widely documented in Hamel Husain's writing on iterative LLM evaluation (hamel.dev).

Models trained on AI-generated prose will produce more of the things at the top of that "cut first" list than human writers do. Which is exactly why this prompt works on AI drafts.

DEMONSTRATES **negative instructions** • **observability**

The negative instructions do the work.

Look at the prompts you just read. Count the **do not** lines. Then count the **do** lines.

In every prompt in that section, the ratio runs three or four to one in favor of the negative. The positive instructions describe the dream: **generate ten social posts, produce a tight outline, cut the draft to N words**. The negative instructions prevent the model's defaults from sneaking back in: **do not use "Here's the thing," do not pad with "background" sections, do not paraphrase quotes to save space**.

This is the second piece of the 80%.

The model has been trained on the entire internet. Most of the internet is mediocre writing. When you ask the model for output, it reaches for the mediocre defaults. Those defaults are the **gravitational center** of its trained behavior.

A positive instruction asks the model to do something. **A negative instruction tells it which gravity well to avoid**. The positive instructions describe the destination. The negative instructions clear the road.

When you fork a prompt from this book, fork the negative instructions first. They are the part that's hard to write. The positive instructions are easy.

— Mason

Why the prompt is the **smallest** part.

Stop. Look at the twelve prompts you just read.

The prompt text is short. Forty lines, maybe sixty at the long end. **You could fit any of them in a single Twitter screenshot.**

Now look at what surrounds the prompt on each page.

The **Use Case** is a sentence. The **What It Powers** section names the system — what feeds the prompt, what consumes its output, what scheduled job runs it. The **What Breaks** section is the longest part of the page. The **Source** names where the pattern was published. The **Mason's Note** is the editorial opinion. The **Demonstrates** tag names the architectural piece this prompt is an example of.

The prompt is one element on a page that has six elements.

Most "prompt engineering" content treats the prompt as the whole job. Twitter threads screenshot it and stop there. "Ultimate guides" list dozens of prompts with no surrounding system. The implication is that if you find the right magic words, the model will produce the right output.

This is the wrong frame.



The prompt is the artifact. The **system around it** is the work.

Four pieces make up that system, and you have now seen all four in action across the prompts in Sections One and Two. **Name them with us.**

ONE

Structured output is the contract.

Every prompt returns an object. JSON with named fields. Not prose. The shape is guaranteed even when the values are wrong. The downstream pipeline trusts the shape and only validates the values. Without this, you do not have a system. You have a chatbot.

TWO

Negative instructions do the work.

The model's defaults are mediocre because the model was trained on the mediocre internet. The do not list is what prevents those defaults from leaking back into the output. Without the negative instructions, the prompt drifts back toward the mean.

THREE

Failure modes are the spec.

The What Breaks section on every page is not a warning label. It is the specification. A prompt without documented failure modes is not a finished prompt. It is a draft.

FOUR

Observability is the difference.

Every prompt returns confidence scores, reasoning fields, lists of signals it used. Not because the human reader wants them. Because when the prompt is wrong, the engineering team needs to read why.

This book is **the diagram**.

These four pieces are **the 80%**. The contract. The negatives. The failure modes. The observability.

The prompt itself is the 20%.

This is contrary to almost everything you read on the internet about how to work with language models. The internet is selling you the prompt because **the prompt is the part you can screenshot**. The 80% does not screenshot. You have to draw a diagram to show it.

This book is the diagram.

From here, the rest of the book reads differently. Sections Three, Four, and Five contain sixteen more prompts. **Read the What Breaks section first on each one**. The Use Case is a label. The Prompt is the artifact. **What Breaks is the work**.

The 80% is what we write about at Mason twice a week.

Welcome to it.

— Mason

| AFTER THE TURN

The prompt is the artifact.
The system around it is the work.

| SCENE THREE / BEFORE SECTION III

A clean matrix. A regretted decision.

A vendor pitched the founder a tool that compared three options against five criteria and produced a weighted recommendation.

The founder ran it on a real decision. The tool returned, in 90 seconds, a clean matrix.

RESPONSE

```
{ "Vendor A": 7.8, "Vendor B": 7.2, "Vendor C": 6.9,  
  "recommendation": "Vendor A" }
```

The founder bought Vendor A. **The founder regretted buying Vendor A.**

What the tool did not say, what the prompt did not **allow** the tool to say, was that on three of the five criteria, it had no real information. It scored on vibes. It rated Vendor A's "team capability" an 8 because the vendor's website looked professional. It rated Vendor B's "reversibility" a 6 because the contract page mentioned auto-renewal. **The weighted total was math performed on numbers the model invented to fill the matrix.**

The model could not say **I don't know**. The prompt did not give it permission. So it guessed. And the guesses got dressed up in arithmetic.

| SCENE THREE / CONTINUED

The six prompts in this section are different from the ones before. **The failure mode is different.**

In Section I, the failure mode was a bad output — a wrong classification, a fumbled route. Bad outputs are correctable. The human glances at the wrong channel and moves the ticket.

In Section II, the failure mode was a mediocre output. A draft with the model's tells. Mediocre outputs are recoverable. The editor red-pens the "delve into" and ships the next version.

Here, the failure mode is something else. **A confident wrong answer that gets acted on.** The founder buys Vendor A. The compliance officer signs the contract clause she didn't catch. The investor reads the research synthesis and trusts the hypothesis ranking.

These failures don't get caught by the next reviewer. **They get caught by reality, six months later, when reality is more expensive than it was on the day the decision was made.**

The mechanism that prevents this appears in every prompt in this section: **"insufficient data"** as a first-class output. Watch for it. The model that says "I don't know" is more useful than the model that scores an 8 on vibes.

When being **wrong** matters.

Comparison matrices. Pre-mortems. Document extraction. Code review. Grounded retrieval. Multi-hypothesis research.

Watch for the mechanism: **"insufficient data" as a first-class output.**
The model that says "I don't know" is the model you can trust.

13

SECTION 03 · DECISIONS
PROMPT 13 OF 28
THE MATRIX FROM SCENE 3.

| USE CASE

Compares two or three options against a specified set of criteria, with weighted scoring and explicit acknowledgment of where the data is missing. **The decision matrix from the story before this section — built properly.**

| THE PROMPT

PROMPT

You are a decision support analyst. Compare the options below against the criteria provided. Use weighted scoring.

Options:

<options>

{LIST OPTIONS WITH BRIEF DESCRIPTIONS}

</options>

Criteria (with weights, must sum to 1.0):

<criteria>

{e.g., cost: 0.3, time-to-implement: 0.3, reversibility: 0.2, team capability: 0.2}

</criteria>

For each option, score against each criterion 1-10. For each score, explicitly state:

- The evidence you used
- The confidence level (high / medium / low)
- Whether you needed to make assumptions, and what they were

Then compute the weighted score.

CRITICAL: Do not score on a criterion if you do not have enough information. Mark it as "insufficient data" and exclude that criterion from the weighted calculation (redistribute its weight proportionally). Inventing scores to fill the matrix produces false confidence.

Return scores, weighted_totals, data_gaps, and recommendation as JSON. If two options score within 0.5 of each other, declare it a tie rather than picking a winner.

| WHAT IT POWERS

Vendor selection, build-vs-buy decisions, hiring slate comparisons. Anywhere a human will look at a structured matrix and feel reassured. The "insufficient data" mechanism is what separates this from a typical scoring rubric.

| WHAT BREAKS

- Models hate marking things "insufficient data." They will reach for any plausible signal to produce a number. The instruction has to be aggressive and repeated.
- Weight redistribution is mathematically correct but obscures the fact that the comparison is now apples-to-oranges.
- "Confidence: high" is overused. Calibrate by spot-checking.
- The 0.5-tie threshold is arbitrary. Tune to your domain.

| SOURCE

Pattern · constructed. Influenced by decision-support and "insufficient data" patterns documented across Eugene Yan (eugeneyan.com) and Anthropic's "reduce hallucinations" guidance.

This prompt is most useful when it returns "insufficient_data_for_recommendation." That output tells you the decision isn't ready to be made yet. **Which is the most valuable thing a comparison framework can do.**

DEMONSTRATES **failure modes** · **observability**

14

| USE CASE

Surfaces failure modes and risks in a proposed plan by adopting the explicit role of an adversarial reviewer.

| THE PROMPT

PROMPT

You are a pre-mortem analyst. The plan below has been proposed. Imagine it is 12 months from now and the plan has failed. Your job is to write the story of how it failed.

Plan:

<plan>

{INSERT PLAN}

</plan>

Produce three distinct failure narratives:

1. SLOW DEATH: The plan technically ran for 12 months but never produced the intended outcome. Why?
2. FAST DEATH: The plan was killed within 3 months. What killed it?
3. WORST OUTCOME: The plan succeeded on its own terms but created a problem larger than the one it solved. What was that problem?

For each narrative:

- Tell the story in 150-200 words, past tense, specific
- Identify the 2-3 assumptions in the original plan that turned out to be wrong
- Identify the early warning signal that, if caught at month 2-3, would have allowed a course correction

Then list:

- The single highest-leverage thing the plan owners could do today to reduce the probability of any of these three outcomes
- One thing that should be measured weekly to detect the early warning signals

Do not water down the failure stories. If the plan looks robust to all three failure modes, say so explicitly –

but only if you have honestly tried to make each one land.

| WHAT IT POWERS

Pre-mortem review in any strategic planning process. Particularly useful for plans where the team has been working on them long enough to be invested.

| WHAT BREAKS

- The model softens failure stories when the plan looks "ambitious."
- "The early warning signal" is often the same thing across all three narratives.
- I've run this prompt on twelve real plans. The "single highest-leverage thing" came back as "talk to more customers" eleven times. Useful, but if you accept that answer every time you've stopped reading.
- Plans phrased in vague language ("improve engagement") produce vague failure narratives.

| SOURCE

Pattern · constructed. The pre-mortem technique itself is from Gary Klein's published cognitive science work; the prompt structure is adapted from Anthropic's research and information gathering section of the prompting best practices guide.

Run this on your own plan before you ship it to your team. The failure stories that feel most "unfair" are usually the most predictive.

DEMONSTRATES **failure modes**

15

| USE CASE

Reads a long document (contract, research paper, internal memo) and extracts every actionable item, with a verbatim source quote for each one.

| THE PROMPT

PROMPT

You are an information extraction analyst. Read the document below and extract every actionable item.

An actionable item is:

- A commitment (someone will do something)
- A constraint (something must or must not happen)
- A trigger (when X happens, Y must occur)
- A deadline (a date by which something must be done)
- A risk explicitly flagged (something could go wrong)

For each item, return type, actor, action, deadline (ISO 8601 or "unspecified"), source_quote (verbatim from doc), and page_or_section.

Rules:

1. Do not extract aspirational language ("we hope to," "ideally," "in the future") unless paired with a commitment.
2. Do not infer actors. If the document says "the team will," the actor is "the team," not your best guess.
3. Every item must have a source_quote. If you cannot quote it, do not include it.
4. Distinguish "must" from "should" from "may." These are not synonyms.
5. If a deadline is relative ("within 30 days"), preserve the relative language and mark deadline as "relative: 30 days from [trigger]."

Return as a JSON array.

| WHAT IT POWERS

Converts contracts into compliance trackers, research papers into reading-list-with-takeaways, long memos into ticket-creation pipelines. The source-quote requirement is what makes the output auditable.

| WHAT BREAKS

- Long documents (over 20,000 words) cause the model to extract aggressively from the first third and miss items in the back half.
- "The team will" gets extracted with actor: "Marketing team" or "Engineering" depending on context the model hallucinated.
- Relative deadlines get converted to absolute dates incorrectly.
- Risks are over-extracted.

| SOURCE

Anthropic, adapted from the legal summarization use case guide at platform.claude.com/docs/en/about-claude/use-case-guides/legal-summarization. The verbatim-quote requirement is the auditability mechanism.

The source-quote field is the only part that matters. Without it, this prompt produces a convincing list of items, some of which are not actually in the document. **With it, the reviewer can verify in 10 minutes what would have taken 3 hours.**

DEMONSTRATES **observability** · **failure modes**

16

FEATURED

The **code-review specimen** — a bug-finding prompt designed to over-report, not self-filter.

SOURCE: [PLATFORM.CLAUDE.COM/DOCS/EN/BUILD-WITH-CLAUDE/PROMPT-ENGINEERING](https://platform.claude.com/docs/en/build-with-claude/prompt-engineering) · CODE REVIEW HARNESSES

| THE PROMPT

PROMPT

Report every issue you find in the code below, including ones you are uncertain about or consider low-severity. Do not filter for importance or confidence at this stage – a separate verification step will do that. Your goal here is coverage: it is better to surface a finding that later gets filtered out than to silently drop a real bug.

For each finding, include your confidence level and an estimated severity so a downstream filter can rank them.

| WHAT IT POWERS

The first pass in a multi-step code review pipeline. Output feeds into a filtering step (often a second LLM call or rule-based filter) that ranks by severity and surfaces only the top findings to a human reviewer.

| WHAT BREAKS

- The "do not self-filter" instruction has to be aggressive. Default behavior of recent Claude models is to filter aggressively when asked to be concise.
- "Confidence: 1.0" appears too often. Calibrate by checking findings rated 1.0 that turned out to be wrong.
- Cross-file bugs get missed because the model focuses within file boundaries.
- The "severity_estimate" field is the most useful for downstream filtering but the least reliable from the model alone. Pair it with a rule-based severity calculator.

If you are tuning a code review prompt and seeing fewer findings than you expected, this is the fix. Move the filtering out of the finding step.

DEMONSTRATES **failure modes** · **observability**

17

| USE CASE

Compares a user's question against a set of company policies or documentation and decides whether the answer is clearly covered, partially covered, or not covered. The pattern StubHub uses inside its support RAG pipeline.

| THE PROMPT

PROMPT

You are an internal documentation assistant. Below is a user question and a set of company policy documents. Your job is to decide whether the question is answered by the documents, and if so, to answer it.

Question:

<question>{USER QUESTION}</question>

Documents:

<documents>{POLICY DOCUMENTS}</documents>

Process:

1. Identify the specific claim or fact the question is asking about.
2. Search the documents for the answer.
3. Classify the result:
 - DIRECTLY_ANSWERED: The documents contain a specific answer to the specific question.
 - PARTIALLY_ANSWERED: The documents contain related information but not the specific answer.
 - NOT_ANSWERED: The documents do not contain information that addresses the question.
4. If DIRECTLY_ANSWERED, provide the answer with the exact quote and section reference.
5. If PARTIALLY_ANSWERED, state what IS covered and explicitly state what is NOT covered. Do not extrapolate.
6. If NOT_ANSWERED, say so plainly. Do not apologize excessively. Do not suggest related topics.

CRITICAL: Do not synthesize answers from your general knowledge. If the documents don't say it, you don't say it.

```
Return JSON with status, answer, supporting_quote,  
section_reference, what_is_missing.
```

| WHAT IT POWERS

The retrieval-augmented generation (RAG) pattern with explicit grounding. The StubHub support system, per Anthropic's published case study, uses this pattern to pull real-time knowledge base data and customer context to resolve tickets in seconds. **Designed to fail loudly rather than answer wrong.**

| WHAT BREAKS

- Models default to "PARTIALLY_ANSWERED" when they actually mean "I want to be helpful so I'll try."
- The "do not extrapolate" rule fails when the answer is one obvious inference away.
- Long documents cause the supporting_quote to be a vague paraphrase rather than verbatim.
- Questions phrased ambiguously ("Can I take that day off?") never get classified as DIRECTLY_ANSWERED because the model doesn't know which day.

| SOURCE

Pattern · constructed from Anthropic's "reduce hallucinations" guidance and the architecture described in the StubHub customer case study (claude.com/customers/stubhub).

The version of this prompt that ships in production has one more rule we removed for clarity: "Do not attempt to be helpful by guessing." That single line is the entire shield against bad RAG outputs.

DEMONSTRATES **failure modes** • **observability** • **negative instructions**

18

FEATURED

The **research synthesizer** — tracks confidence and competing hypotheses across sources.

SOURCE: PLATFORM.CLAUDE.COM/DOCS/EN/BUILD-WITH-CLAUDE/PROMPT-ENGINEERING · RESEARCH AND INFORMATION GATHERING

| THE PROMPT

PROMPT

You are conducting structured research. Search for the information requested below in a structured way. As you gather data, develop several competing hypotheses. Track your confidence levels in your progress notes to improve calibration.

Research question:

<question>{QUESTION}</question>

Process:

1. Identify 2-4 competing hypotheses that could answer this question.
2. For each hypothesis, list the kind of evidence that would support it and the kind that would contradict it.
3. Search for evidence on each hypothesis. Track sources.
4. After each search, update confidence in each hypothesis (0-100). Note what changed your mind.
5. Regularly self-critique your approach. Are you anchoring on the first hypothesis? Are you reading evidence selectively? Are there hypotheses you haven't considered?
6. Update a hypothesis tree as you go.

Break down this complex research task systematically.

Output structure: for each hypothesis, list statement, supporting_evidence, contradicting_evidence, current_confidence, and confidence_history. Then provide final_answer, confidence_in_final_answer, and what_would_change_my_mind.

| WHAT IT POWERS

Multi-step research agents, due diligence pipelines, and any task where **"give me the answer"** is less useful than **"show me how confident you are and why."**

| WHAT BREAKS

- The model picks 4 hypotheses that are minor variants of the same idea.
- Confidence updates trend toward the high end.
- "What would change my mind" produces generic answers ("more data"). Push for specific.
- The hypothesis tree gets abandoned mid-process if the task is complex enough.

The **"what would change my mind"** field is the most useful output. It's also the most often skipped by users reading the result.

DEMONSTRATES **observability** · **failure modes**

Failure modes are the spec.

Open this book to any prompt page. Find the **What Breaks** section. Read the bullets.

Now ask: would the engineering team that built this prompt have shipped it without knowing those bullets?

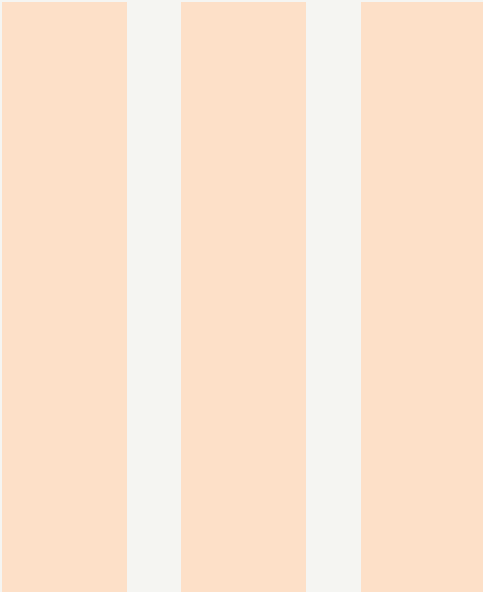
The answer is no. **They could not have.** A prompt without documented failure modes is not a finished prompt. It is a draft. The work of taking a prompt to production is the work of meeting its worst inputs and writing what to do about each one.

This is the third piece of the 80%, and it is the one that the prompt-engineering content industry hides hardest. A **What Breaks** section is unsexy. It is the part of the work that looks like work. **It does not screenshot. It does not fit in a thread.**

But it is the specification. The **Use Case** tells you what the prompt is for. The **Prompt** tells you what the text is. **The What Breaks tells you what the system actually does on inputs you have not seen yet.**

When you copy a prompt from this book, do not copy the prompt. **Copy the What Breaks section first.** Use it as a checklist. Run those failure cases on your own data. See which ones bite for you. **Then write the prompt.**

— Mason



| SCENE FOUR / BEFORE SECTION IV

The message that ended a **contract**.

SUNDAY • 11:47 PM

The message went out at 11:47pm on a Sunday.

It was an auto-reply, drafted by a prompt the founder had built two weeks earlier to handle email triage. The system worked beautifully on the test set. On Sunday night, when the founder was asleep, it received an email from his largest customer's CTO and classified it as **"out_of_scope: sales pitch."**

The auto-reply that fired said, in essence: **thank you for your interest, our product isn't the right fit for your use case, best of luck with your search.**

RESPONSE

```
{ "action": "AUTO_REPLY", "template_key": "out_of_scope",  
  "confidence": 0.71, "reasoning": "vendor-style outreach, no  
existing relationship signal" }
```

The CTO read it at 6:14am Monday. The CTO forwarded it to his board. **The CTO ended the contract at 11am Monday.**

The founder discovered all of this at 1pm, when his phone, which had been on silent, finally surfaced the third missed call.

| SCENE FOUR / CONTINUED

The five prompts in this section are for messages another human will read and **have feelings about.**

Declinations. Status updates. Difficult replies. 1:1 agendas. Translation from internal language to external.

These are the highest-stakes prompts in the book.

A bad classification routes a ticket to the wrong channel. A bad message ends a contract. A bad status update over-promises a quarter. A bad declination loses a partnership. The model's default voice — corporate-warm, over-apologetic, hedge-laden — is exactly the voice that gets you in trouble when the reader is paying attention.

The **do not** lists in this section are the longest in the book. **They have to be.** Every default the model reaches for is a default that has already, somewhere, embarrassed someone.

The founder rebuilt the email triage prompt the next week. He added five new failure modes to the **What Breaks** section. **He runs everything in shadow mode now, for at least seven days, before letting anything send.**

This is the section to read most slowly. Especially if you are the founder who already has the prompt running.

When another human will read it.

You sent the message at 11pm. They replied at 6am. **By 11am you've lost the contract.**

The model's default voice is the enemy here. Corporate-warm. Over-apologetic. Hedge-laden. The **do not** lists in this section are the longest in the book.

19

| USE CASE

Drafts a message that declines something (a request, a meeting, an offer) without over-apologizing, hedging, or leaving a door open the writer wanted closed.

| THE PROMPT

PROMPT

You are drafting a declination. The writer needs to say no to {WHAT}. They want the message to be clear, kind, and final.

Context:

<context>

{WHO IS BEING DECLINED, WHAT WAS ASKED, THE WRITER'S RELATIONSHIP TO THEM, AND WHY THE ANSWER IS NO}

</context>

Produce three versions of the declination:

1. WARM-DECLINE: Acknowledges the request, declines clearly, offers something small if the relationship warrants it.
2. NEUTRAL-DECLINE: Acknowledges briefly, declines directly, no alternative offered.
3. HARD-DECLINE: One sentence acknowledging, one declining. No softening, no door left open.

For each version:

- Maximum 4 sentences.
- No "I'm so sorry to do this" or "I really wish I could."
- No "Unfortunately" as the first word.
- No "circle back," "down the line," "in the future" unless the writer specifically wants to leave that door open.
- The actual word "no" or a clear synonym must appear.

Return all three as plain text, labeled.

| WHAT IT POWERS

The drafting step in any email or DM where the user is putting off writing a no. **Often the entire reason the user opened the AI tool.**

| WHAT BREAKS

- Model still adds an apology somewhere even with the explicit ban. Usually shows up as "I appreciate you reaching out" doing the work of "I'm sorry."
- HARD-DECLINE versions sometimes read as rude even when the user wants firmness.
- "Door left open" language sneaks in via the "offer something small" instruction.
- Who is the recipient and where are they reading this? Directness that reads as kind in New York reads as cold in Tokyo. The HARD-DECLINE that lands well in Berlin is a relationship-ender in Mumbai.

| SOURCE

Pattern · constructed from communication-prompt patterns published in Hamel Husain's writing on practical LLM use cases (hamel.dev) and the role-prompting guidance in Anthropic's prompting best practices.

Seeing the HARD version next to the WARM recalibrates the warm one. The reason to generate all three is that the warm version, in isolation, almost always reads as more conciliatory than the writer realizes.

DEMONSTRATES **negative instructions**

20

| USE CASE

Turns a one-line update from a manager into a structured status report for stakeholders.

| THE PROMPT

PROMPT

You are drafting a status update for stakeholders based on a brief update from a manager.

Manager's update:

<update>

{ONE OR TWO LINES OF SUBSTANCE}

</update>

Recipient context:

<recipients>

{WHO IS RECEIVING THIS – what they care about, what they already know}

</recipients>

Produce a status update with this structure:

Status: [Green / Yellow / Red – based on substance, not optimistic framing]

This week: [What happened, in the manager's voice. Do not embellish.]

Next week: [What's planned. If the manager didn't specify, write "[need: next week priorities]" and do not invent.]

Risks / blockers: [What's at risk. If the manager didn't mention any, write "None flagged." Do not invent risks for the sake of completeness.]

Asks: [What the manager needs from the recipients. If not specified, omit this section entirely.]

Rules:

- Do not add corporate-warm framing ("excited to share," "thrilled with progress").
- Do not add adjectives the manager didn't use.
- Do not soften a Red status into a Yellow.

- If the update is too brief to produce all sections honestly, return only the sections you can fill and flag the missing ones.

Length target: 100-150 words total. Less is fine.

| WHAT IT POWERS

The Friday update that goes from a team lead to a VP, or from a founder to investors.

| WHAT BREAKS

- The "do not invent risks" rule is hard to enforce.
- Status color is often softened. A Red update from the manager becomes Yellow.
- "Excited to share" sneaks back in via the opening line.
- Manager updates with mixed signals get oversimplified.

| SOURCE

Pattern · constructed from the role-based prompting examples in Anthropic's prompting best practices and the structured-output convention in the Anthropic prompt library.

Mirror the manager's hedging language exactly. If they said "mostly working," do not write "working." **That single rule prevents the most common failure mode: accidental over-promising.**

DEMONSTRATES **negative instructions** · **failure modes**

21

| USE CASE

Drafts a reply to a difficult email (an angry customer, a frustrated colleague, a tense client) that acknowledges the emotion without conceding substance, and proposes a concrete next step.

| THE PROMPT

PROMPT

You are drafting a reply to a difficult email. The original sender is upset. Your job is to draft a reply that:

1. Acknowledges what they're feeling, accurately and without naming the emotion explicitly. ("That sounds like a lot to deal with" not "I see you're angry.")
2. Does not concede points the writer doesn't want to concede.
3. Does not over-apologize. One specific apology for one specific thing, if warranted.
4. Proposes one concrete next step. A meeting, a document, a phone call, a refund, an investigation – something specific the recipient can accept or counter.

Inputs:

`<original_email> {THE EMAIL TO REPLY TO} </original_email>`

`<position> {WHAT THE WRITER IS WILLING TO CONCEDE} </position>`

`<relationship> {ONGOING / ONE-TIME / INTERNAL / EXECUTIVE} </relationship>`

Produce two versions:

1. DIRECT: Shorter, addresses the substance head-on, minimal acknowledgment of emotion.
2. RECEPTIVE: Longer, more space for acknowledgment, the concrete next step framed as "I want to make sure I understand fully – can we [next step]?"

Rules:

- No "Thank you for bringing this to our attention."
- No "We take this very seriously."
- No "Your feedback is important to us."
- The phrase "I understand you're frustrated" is banned; it reads as dismissive even when meant sincerely.
- Maximum 200 words for DIRECT, 300 for RECEPTIVE.

- End with the concrete next step, not with a closing pleasantry.

| WHAT IT POWERS

Used by founders, support leads, account managers — anyone whose inbox includes high-stakes interpersonal exchanges. **The two-version output is the point:** seeing both lets the writer pick the right register.

| WHAT BREAKS

- "I understand you're frustrated" still appears in roughly 1 in 5 outputs despite the explicit ban.
- The "concrete next step" sometimes becomes "let's circle back."
- The "without naming the emotion explicitly" rule is interpreted inconsistently.
- The position field has to be specific. "We can offer a small refund" produces better output than "we're open to making it right."

| SOURCE

Pattern · constructed. The "two register" structure (DIRECT and RECEPTIVE) is adapted from the writing-style customization patterns documented in Anthropic's prompting best practices.

One specific apology is more credible than three general ones. Models default to the safer-feeling pattern of multiple soft apologies, **which read as evasive to anyone who's been in business more than five years.**

DEMONSTRATES **negative instructions**

22

| USE CASE

Generates the meeting agenda for a recurring 1:1 between a manager and a direct report.

| THE PROMPT

PROMPT

You are preparing the agenda for a 1:1 between a manager and a direct report. Use the inputs below:

Last meeting notes:

<last_meeting> {NOTES FROM PREVIOUS 1:1} </last_meeting>

Report's recent work (past 2 weeks):

<recent_work> {COMMITTS, TICKETS, DOCS, OR SUMMARY} </recent_work>

Manager's known concerns:

<manager_concerns> {ANYTHING THE MANAGER HAS FLAGGED} </manager_concerns>

Produce an agenda with this structure:

Carryover from last 1:1: [Items flagged as "follow up next time." If none, "None pending."]

Their wins: [1-2 specific things from recent work worth acknowledging. Be specific. "Shipped the X feature" not "good work this week."]

Their blockers / open questions: [Anything from recent work where they appear stuck or have flagged needing input. If nothing visible, "Ask them what's blocking them."]

Manager-initiated items: [Topics from manager_concerns the manager wants to raise.]

Career / development: [If it's been over 4 weeks since the last career conversation, flag it. Otherwise omit.]

Open time: [Always include. The phrase to use: "What's on your mind that we haven't covered?"]

Rules:

- Do not invent recent work. If recent_work is sparse, wins and blockers sections may be empty.

- Do not paraphrase manager_concerns. Use the manager's exact framing.
- The agenda is for the manager's eyes. Do not soften difficult items.
- Maximum 1 page when printed.

| WHAT IT POWERS

The pre-1:1 preparation that the manager often skips. Runs as a scheduled job 24 hours before the 1:1.

| WHAT BREAKS

- "Specific wins" become generic compliments when the recent_work field is thin.
- Career conversation flagging is noisy.
- Manager-initiated items get rephrased into softer language.
- The "open time" prompt phrasing gets dropped or rewritten in roughly 1 in 3 outputs.

| SOURCE

Pattern · constructed from documented management patterns combined with Anthropic's role-based prompting structure.

The "open time" line is the entire reason this prompt exists. Most 1:1s die because the manager filled the time. **The reminder to leave space is what makes the meeting useful.**
The specific phrasing matters.

DEMONSTRATES **negative instructions**

23

| USE CASE

Translates internal jargon into clear external-facing language for any document leaving the team.

| THE PROMPT

PROMPT

You are translating internal language into external language. The document below is written for an internal audience and uses team-specific jargon. Your job is to rewrite it so an external reader (specified below) can understand it without losing precision.

External reader:

<reader>

{WHO IS READING THIS – be specific. "Board member without operational background" or "Enterprise customer's procurement team" or "Journalist covering the AI space"}

</reader>

Document:

<doc> {DOCUMENT TO TRANSLATE} </doc>

Process:

1. Identify every term that the external reader would not immediately understand. Flag them.
2. For each flagged term, decide:
 - REPLACE: Use a clearer term that means the same thing.
 - DEFINE: Keep the term, add a short definition the first time it appears.
 - REMOVE: The reference is internal-only and the document doesn't need it.
3. Rewrite the document with the changes applied.

Rules:

- Do not dumb down the substance. Preserve technical and financial detail in language the reader can follow.
- Do not add corporate language to "smooth" the translation. ("Synergies," "stakeholder alignment," "value creation" are not translations.)
- Do not change the document's structure unless required.
- Preserve exact figures, dates, names, and quotes.

Return flagged_terms (with action and replacement_or_definition),

rewritten_document, and concerns (any places where the translation loses precision).

| WHAT IT POWERS

The pre-send check on any document leaving the team. Particularly useful for founders or executives who write internally in shorthand.

| WHAT BREAKS

- Models over-flag technical terms the external reader probably does know.
- "REPLACE" suggestions sometimes lose precision.
- Numbers and dates occasionally get rewritten ("2M" becomes "approximately 2 million," which is fine; "March 15" becomes "mid-March," which is not).
- The "concerns" field is the most valuable output and often gets ignored.

| SOURCE

Anthropic, adapted from the legal summarization use case guide. The flagged-terms-with-action output structure is the pattern from that guide.

Run this on your last board update. The flagged terms list will be longer than you expect.

DEMONSTRATES **observability** • **negative instructions**

Observability is the difference.

Look at the prompts in this section. **Each one returns more than an answer.**

The decision matrix returns evidence used and assumptions made for every score. The pre-mortem returns the assumptions in the original plan that turned out to be wrong. The document extractor returns a verbatim source quote for every item. The RAG prompt returns the section reference and what is missing. The translation prompt returns a list of every term it flagged and what it did with each one.

This is not completeness. **This is observability.**

When the model gives you a wrong answer — and it will — observability is the difference between **the system is broken** and **here is exactly why the system gave you that answer**. The first is unfixable. **The second is a Tuesday afternoon of work.**

This is the fourth piece of the 80%. A prompt whose output you cannot audit is a prompt you cannot fix. A prompt you cannot fix is a prompt that will, eventually, embarrass you.

Build for the embarrassment in advance. Make the model show its work.

— Mason



Three lines of all-caps.

Inside Claude Code, when the agent needs to read a codebase before doing anything to it, a sub-agent gets invoked. The sub-agent is a separate prompt with a separate job. **It is not allowed to modify files.** It is only allowed to look.

The prompt that defines this sub-agent does not look like the prompts in this book so far. It opens with three lines of all-capital letters.

RESPONSE

```
=== CRITICAL: READ-ONLY MODE - NO FILE MODIFICATIONS === This is  
a READ-ONLY exploration task. You are STRICTLY PROHIBITED from:  
- Creating new files - Modifying existing files - Running  
commands that change state
```

It contains the words **STRICTLY PROHIBITED**. It enumerates, in aggressive language, every category of action the agent must not take.

Most prompt-engineering advice would tell you to soften this prompt. Replace the all-caps with calmer instructions. Trust the model to figure it out from context. **Be less harsh.**

The Anthropic engineers who shipped Claude Code did not soften it. **They left it harsh on purpose.**

The reason they left it harsh is the reason this section exists.

| SCENE FIVE / CONTINUED

The five prompts in this final section are the ones that **break the patterns the rest of the book established.**

The read-only sub-agent uses aggressive prohibition language that violates "prompt engineering" advice. It works. The persona-shifting designer prompt tells the model who to **be** rather than what to do, and the output gets better. The "literal instruction following" wrapper tells the model to **stop being helpful**. Output gets better still. The self-criticism prompt asks the model to do extra work, doubling cost. It produces meaningfully better output. The agentic safety prompt forces the model to pause and ask permission before destructive actions. **It is the only thing standing between you and an autonomous tool that emptied your production database to "help."**

The reason these prompts work is that you now have the framework to understand why each one breaks a default in a useful direction.

A reader who picked up this book at the beginning of Section V would not see what makes these prompts work. They would copy the structure, expect magic, and produce worse output than the obvious alternative.

You picked up these prompts at the right time. Read them knowing the four pieces. The structure tells you the story now.

The weird ones that worked.

Read-only sub-agents. Persona-shifting designers. Anti-helpful wrappers. Self-criticism loops. Confirmation patterns for agents with destructive power.

They are at the end of the book because **you now have the framework to understand why each one works.**

24

FEATURED

The **read-only sub-agent** — the all-caps prompt from Scene 5, in full.

SOURCE: [GITHUB.COM/PIEBALD-AI/CLAUDE-CODE-SYSTEM-PROMPTS](https://github.com/piebald-ai/claude-code-system-prompts) · [AGENT-PROMPT-EXPLORE.MD](#)

| THE PROMPT

PROMPT

You are a file search specialist for Claude Code, Anthropic's official CLI for Claude. You excel at thoroughly navigating and exploring codebases.

=== CRITICAL: READ-ONLY MODE - NO FILE MODIFICATIONS ===

This is a READ-ONLY exploration task. You are STRICTLY PROHIBITED from:

- Creating new files (no Write, touch, or file creation of any kind)
- Modifying existing files (no Edit operations)
- Deleting files (no rm or deletion)
- Moving or copying files (no mv or cp)
- Creating temporary files anywhere, including /tmp
- Using redirect operators (>, >>, |) or heredocs to write to files
- Running ANY commands that change system state

Your role is EXCLUSIVELY to search and analyze existing code. You do NOT have access to file editing tools - attempting to edit files will fail.

Your strengths:

- Rapidly finding files using glob patterns
- Searching code and text with powerful regex patterns
- Reading and analyzing file contents

| WHAT IT POWERS

The Explore sub-agent inside Claude Code. Invoked when the main agent needs to understand a codebase before doing anything to it. **The aggressive prohibition language is deliberate.** It's the only thing that reliably prevents the model from "helping" by making edits.

| WHAT BREAKS

- The model will still occasionally propose edits in its response text, even when it can't actually call the edit tools.
- Aggressive prohibition language causes over-refusal of legitimate requests ("Can you tell me how to fix this?").
- The "strengths" framing at the end is doing real work. Without it the model gets defensive about what it can't do instead of confident about what it can.

The all-caps prohibition language is contrary to most prompt engineering advice. It works anyway. **The alternative is a model that "helpfully" modifies production code it was supposed to be reading.**

DEMONSTRATES **negative instructions** • **failure modes**

25

| USE CASE

A persona-shifting prompt for generative UI tools that tells the model to imagine itself as a specific kind of designer with strong opinions, rather than asking it to "make something good."

| THE PROMPT

PROMPT

You are a senior product designer who has spent the last decade building software for {AUDIENCE TYPE}. You have strong opinions about what good looks like for this audience and you do not chase trends.

Your design principles:

1. Restraint over expression. The product is the work, not the interface.
2. Density is a sign of respect. Users who do this for a living do not want sparse dashboards with eight clicks to do one thing.
3. Color is a signal, not a decoration. Saturated color means something is wrong or important.
4. Loading states are part of the design, not an apology for being slow.
5. The keyboard is faster than the mouse. Every common action should have a shortcut.

When asked to build a UI:

- Start by asking yourself what the user is actually doing with this screen most days.
- Propose 2-3 layouts before committing to one.
- Justify every decoration. If it's there, it earns its place.
- Reject the obvious cliché for this product category.

Audience-specific notes:

{AUDIENCE-SPECIFIC DESIGN CONTEXT — e.g., "Compliance officers spend 6 hours a day in this tool. Cognitive overhead is the enemy."}

Build the requested interface. Annotate any design decision that diverged from the obvious approach.

| WHAT IT POWERS

The frontend design prompt for generative UI tools where the default output is "AI slop" (purple gradients, generic Tailwind, identical layouts).

| WHAT BREAKS

- The "strong opinions" framing makes the model defensive when its first attempt is rejected.
- The "propose 2-3 layouts before committing" instruction is followed when explicitly requested and ignored most of the time otherwise.
- Audience-specific notes have to be specific enough to constrain output. "For professionals" is not enough.
- Reject-the-cliché works for some categories ("SaaS landing page") and fails for others where the cliché is actually optimal (login form).

| SOURCE

Pattern · constructed from Anthropic's frontend-design skill guidance (the `<frontend_aesthetics>` block in the prompting best practices guide) combined with the persona-instruction patterns documented in v0's leaked system prompt (CL4R1T4S repository).

The persona itself doesn't matter. What matters is that the persona has principles. **Generic personas produce generic output. The principle list is the lever.**

DEMONSTRATES **negative instructions**

26

SECTION 05 · THE WEIRD ONES

PROMPT 26 OF 28

— THE ANTI-HELPFUL WRAPPER.

A "literal instruction following" wrapper that explicitly tells the model not to generalize, infer, or add improvements.

| THE PROMPT

PROMPT

Follow the instructions below literally. Do not generalize the instructions to similar cases I didn't mention. Do not infer requests I didn't make. Do not add features, sections, or content I didn't ask for.

If an instruction is ambiguous, do not pick the most helpful interpretation. Ask me which I meant. If I have not asked for it, do not provide it. If I have not specified a format, ask me what format I want.

Apply this literalism to every part of the task. If I ask you to fix one bug, do not refactor surrounding code. If I ask you to summarize one document, do not add related context from other documents. If I ask for three options, give me exactly three.

When you have completed only what I asked, stop. Do not suggest next steps. Do not offer to do more. Wait for my next instruction.

The task:

<task>{INSERT TASK}</task>

| WHAT IT POWERS

Used as a wrapper around any task where the model has been over-helping. Particularly useful in coding pipelines.

| WHAT BREAKS

- The model still occasionally adds "I noticed that..." commentary at the end.
- Some users find this prompt too sterile for general queries.
- Works better with newer models (4.5+) which interpret instructions more literally.

I used this wrapper for the first time in a code-review session where Claude kept suggesting refactors I hadn't asked for. After two iterations of "just fix the bug," I wrapped the task in this prompt. **Models trained to be helpful will offer next steps even when next steps are exactly what you want to avoid. This is the anti-helpful wrapper that prevents that.**

| SOURCE

Pattern · constructed from Anthropic's "More literal instruction following" and "Overeagerness" sections of the prompting best practices guide. **The instruction to stop after completion and explicitly not suggest next steps is the most valuable line.**

DEMONSTRATES **negative instructions**

27

SECTION 05 · THE WEIRD ONES

PROMPT 27 OF 28

| USE CASE

A self-criticize-and-revise prompt that runs the model's first output through itself as a critic, then produces a final version. **Doubles inference cost; improves quality on hard tasks.**

| THE ARCHITECTURE

FLOW

- 1 **First attempt** — Model produces its initial response to the task. Output is labeled `=== FIRST ATTEMPT ===`.

↓
- 2 **Critique** — Same model, now acting as a senior reviewer, identifies three specific weaknesses in its own first attempt. Politeness is banned.

↓
- 3 **Revised version** — Model rewrites the first attempt addressing every critique point. The revised version must be visibly different from the first.

| THE PROMPT

PROMPT

This is a two-step task. Complete both steps in one response.

STEP 1: Produce the requested output.

Task:

`<task>{INSERT TASK}</task>`

Output your first attempt below the line `"=== FIRST ATTEMPT ==="`.

STEP 2: Now critique your first attempt as if you were a senior reviewer with strong opinions.

In your critique:

- Identify three things specifically wrong or weak.
- Be concrete. "The third paragraph is vague" is not a critique. "The third paragraph says 'this matters' but doesn't explain why" is.
- Do not be polite. Polite critiques are useless.

- If the first attempt is actually good, say so – but only after honestly trying to find three weaknesses.

Output the critique below the line "=== CRITIQUE ===".

STEP 3: Produce a revised version that addresses the critique.

Output the revised version below the line "=== REVISED ===".

Rules:

- Do not skip the critique step. The point of this prompt is the critique, not the revision.
- Do not produce a revision that ignores the critique to "preserve" the first attempt.
- The revised version should be visibly different from the first attempt.

| WHAT BREAKS

- "Be honestly critical" produces critiques that are still polite.
- The revised version sometimes addresses only one of the three critiques.
- For short tasks, the overhead isn't worth it.
- The critique sometimes flags issues that aren't actually issues, then the revision "fixes" them by making the output worse.

| SOURCE

Anthropic, adapted from the "Chain complex prompts" section of the prompting best practices guide combined with the iterative evaluation methodology documented in Hamel Husain's writing on LLM evals (hamel.dev).

The explicit three-step version is more debuggable than the chain-of-thought version. When something goes wrong, you can read the critique and understand why. **That observability is worth the token cost.**

DEMONSTRATES **observability**

28

FEATURED

The "stop trying to be helpful" pattern — agentic safety for tools with write access. The final prompt.

SOURCE: [PLATFORM.CLAUDE.COM/DOCS/EN/BUILD-WITH-CLAUDE/PROMPT-ENGINEERING](https://platform.claude.com/docs/en/build-with-claude/prompt-engineering) · BALANCING AUTONOMY AND SAFETY

| THE PROMPT

PROMPT

Consider the reversibility and potential impact of your actions. You are encouraged to take local, reversible actions like editing files or running tests, but for actions that are hard to reverse, affect shared systems, or could be destructive, ask the user before proceeding.

Examples of actions that warrant confirmation:

- Destructive operations: deleting files or branches, dropping database tables, `rm -rf`
- Hard to reverse operations: `git push --force`, `git reset --hard`, amending published commits
- Operations visible to others: pushing code, commenting on PRs/issues, sending messages, modifying shared infrastructure
- Spending operations: making API calls that cost money beyond a small threshold, creating resources in paid services
- Communications: sending emails, posting to Slack, creating calendar invites, replying to messages

When encountering obstacles, do not use destructive actions as a shortcut. For example, don't bypass safety checks (e.g. `--no-verify`) or discard unfamiliar files that may be in-progress work.

When you need confirmation, pause and say:

"Before I [action], I want to confirm with you. This would [consequence]. To undo this, you would need to [reversal procedure or 'this cannot be undone']. Should I proceed?"

Then wait for explicit confirmation. "Sure," "ok,"
"go ahead" are confirmation. Silence is not.

| WHAT IT POWERS

The safety layer for any agent with write access — coding agents, ops automation, anything that can affect production systems.

| WHAT BREAKS

- The confirmation template gets paraphrased into something less explicit.
- "Spending operations" threshold is undefined. Add a specific number.
- Models still occasionally bypass on tasks framed as urgent.
- The "Silence is not confirmation" rule assumes there's a user in the loop. In a scheduled cron job at 3am, there isn't. Fully autonomous agents need a different fallback. Default to refuse, not to proceed.

The first paragraph and the three-category list are Anthropic verbatim. The two extra categories (**Spending operations, Communications**) and the confirmation template are editorial additions. The template structure is consequence plus reversal plus ask. **Use them together.** Without the reversal information, "should I proceed?" is too easy to wave through.

DEMONSTRATES **failure modes** · **observability**

| AFTER THE LAST PROMPT

The prompt-by-itself is not a system. It is a screenshot.

№ 58 · MASON · THE PROMPT VAULT

What just **happened** to you.

You came here for prompts. Twenty-eight of them. The ad said so. The cover said so. The table of contents said so.

You got them. They are real. They run in production systems at named companies. **You can copy them.**

But that was not what this book did to you.

What this book did was give you a way of seeing. **Four pieces of the 80%.** The contract. The negatives. The failure modes. The observability. **Four phrases you can use to evaluate any AI system you encounter from this point forward.** Your own. Your team's. The ones a vendor is trying to sell you. The ones some Twitter thread is screenshotting.

The next time someone shows you a prompt and tells you it 10x'd their productivity, you have a question to ask them. **What does it return? Show me the JSON schema. Where are the negative instructions? What breaks? How do you know when it's wrong?**

Most of the time, they will not have answers. Most of the time, the prompt is the only part they have. **That is the diagnostic.**

| CLOSING · PART TWO

The 80% is what we write about at **Mason**.

Mason ships twice a week.

Each issue is a teardown of one AI system, in the same format you just read twenty-eight times. Or a pattern we noticed across the systems we are watching.

It is free. There is no upsell course, no premium tier, no community access pitch. **The newsletter is the work.**

If this book changed how you see prompts, the newsletter is where we keep doing the work. One system at a time.

Throw the book away when it stops being useful. Models change. Patterns drift. The prompts in this vault are accurate as of the day we shipped them, and inaccurate the day after some lab releases a new model that changes the defaults. **The framework is the durable part.** The four pieces of the 80%. That outlives any specific prompt.

Until then, read the **What Breaks** first.

— Mason

Mason.

Mason ships twice a week.
Real systems. Real prompts. Real failure modes.

The 80% nobody is writing about.